MICROSOFT FABRIC · PROJET DE BOUT EN BOUT · GUIDE PAS À PAS

RailPulse

De la donnée brute au modèle sémantique : la plateforme de données d'un réseau ferroviaire européen, construite en onze étapes dans un seul espace Fabric.

Source Spark › **Bronze** Copy › **Silver** Dataflow · Spark › **ML** MLflow › **Gold** T-SQL › **Modèle** Direct Lake + **Temps réel** KQL

1 164 305 t de CO_2 évitées en douze mois face à l'avion, calculées train par train.

98 485	26,3 M	1,05 Md€	32
trains certifiés	voyageurs	chiffre d'affaires	mesures DAX prêtes

01 Espace et dossiers
02 Lakehouses et entrepôt
03 Système source simulé
04 Dataflow Gen2
05 Pipeline et Copy
06 Spark Silver certifié
07 Machine learning MLflow
08 Entrepôt Gold en étoile
09 Orchestration complète
10 Flux temps réel KQL
11 Modèle sémantique
12 Contrôle et arrêt

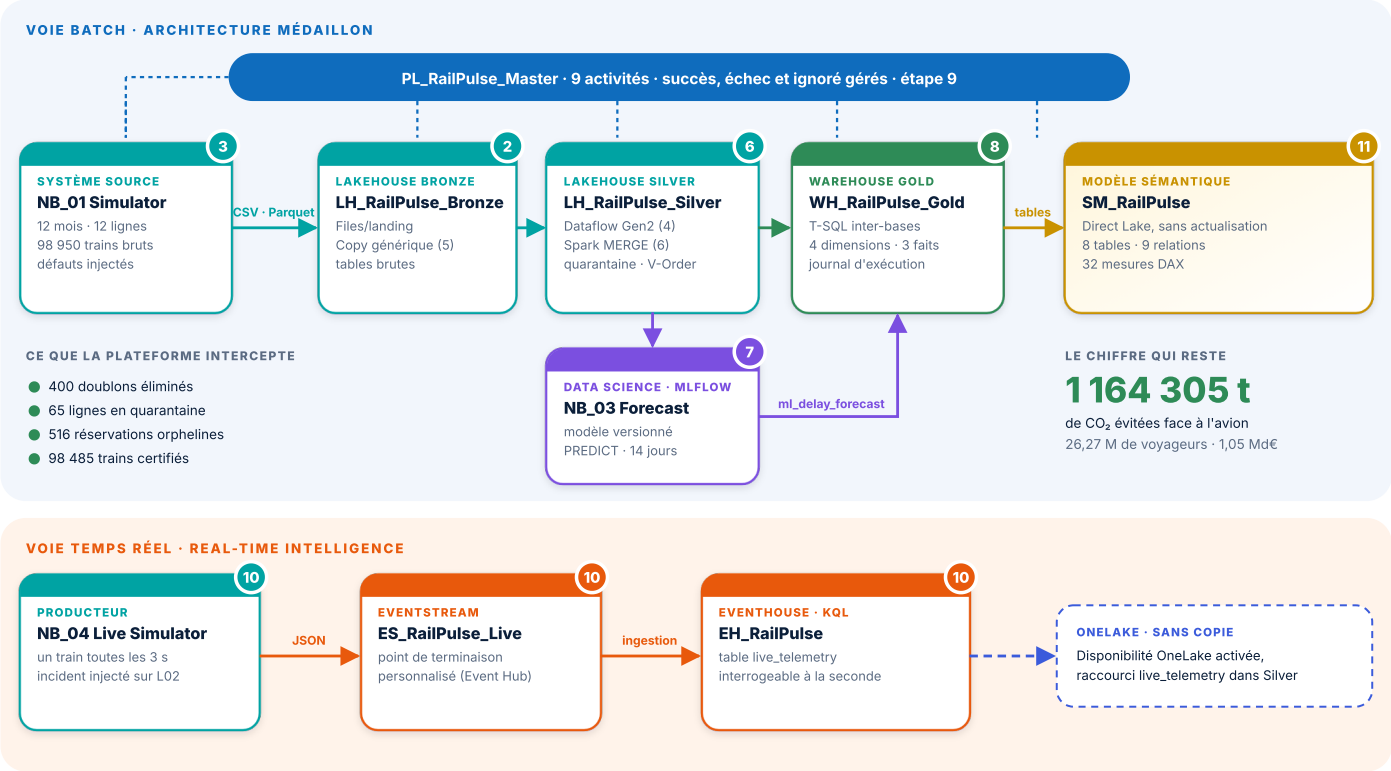
Abdoul Hamid Diallo · Microsoft Data & AI Engineer

15 éléments Fabric · 12 étapes · capacité d'essai

VUE D'ENSEMBLE

Ce que vous allez construire

RailPulse est un opérateur ferroviaire européen fictif : 12 lignes à grande vitesse, 12 gares, 6 pays, de Paris à Amsterdam, Zurich et Milan. En onze étapes, vous bâtissez sa plateforme de données dans un seul espace de travail Fabric : douze mois d'historique traités en architecture médaillon, un modèle de machine learning qui prévoit les retards des quatorze prochains jours, un flux temps réel des trains en circulation, et un modèle sémantique Direct Lake prêt pour Power BI. Chaque pastille numérotée du schéma renvoie à l'étape qui la construit.



Data Factory

Pipeline maître, Copy générique, Dataflow Gen2, procédure stockée, Outlook, gestion succès, échec et ignoré.

Data Engineering

Spark, Delta Lake, MERGE idempotent, quarantaine qualité, intégrité référentielle, V-Order.

Data Warehouse

T-SQL inter-bases sans copie, CTAS, procédure paramétrée, schéma en étoile, journal d'exécution.

Data Science

Expérience MLflow, modèle enregistré et versionné, score par lots avec PREDICT.

Real-Time Intelligence

Eventstream sur point de terminaison Event Hub, Eventhouse, KQL, disponibilité OneLake.

Modèle sémantique

Direct Lake, schéma en étoile, table de dates, 32 mesures DAX créées en une seule requête.

AVANT DE COMMENCER

Les règles du jeu

Ce qu'il vous faut

1. Une capacité Fabric active, l'essai gratuit de 60 jours suffit. Aucun Copilot n'est utilisé.
2. Un compte Microsoft 365 pour l'activité Outlook du pipeline.
3. Les sept fichiers livrés avec ce guide :

FICHIER	NOM
4 notebooks à importer	`NB_01 à NB_04 (.ipynb)`
Entrepôt Gold	`WH_RailPulse_Gold.sql`
Contrôles temps réel	`KQL_RailPulse.kql`
Les 32 mesures	`SM_RailPulse_Mesures.dax`

Importer un notebook

Dans l'espace de travail : **Importer → Notebook → Depuis cet ordinateur**. Le guide explique ce que fait chaque notebook ; le code complet, commenté, est dans le fichier.

Conventions de nommage

ÉLÉMENT	MODÈLE
Lakehouse · entrepôt	`LH_RailPulse_<Couche>` · `WH_RailPulse_Gold`
Notebook	`NB_<ordre>_<Rôle>`
Pipeline · Dataflow	`PL_…` · `DF_…`
Activités du pipeline	`NB_` · `CP_` · `DF_` · `SP_` · `MAIL_` · `FAIL_`
Temps réel · modèle	`EH_` · `ES_` · `SM_`

Les chiffres que vous devez retrouver

OBJET OU INDICATEUR	ATTENDU
bronze_trips · bronze_bookings	**98 950 · 781 102**
silver_trips · silver_quarantine	**98 485 · 65**
silver_bookings · silver_weather	**780 586 · 4 548**
silver_stations · silver_trainsets	**12 · 40**
silver_timetable_plan · ml_delay_forecast	**3 780 · 3 780**
dim_date · dim_route	**457 · 24**
fact_trip · fact_booking	**98 485 · 780 586**
Voyageurs (12 mois)	**26 273 478**
Chiffre d'affaires	**1 049,78 M€**
Ponctualité à 5 minutes	**75,56 %**
CO_2 évité face à l'avion	**1 164 305 t**
Trains à risque élevé (14 jours)	**environ 400**

Le générateur utilise une graine fixe : tout le monde obtient les mêmes volumes. Seul le nombre de trains à risque peut varier de quelques unités selon la version des bibliothèques de Spark.

L'ordre de construction

Chaque brique est construite et testée seule, puis le pipeline de l'étape 9 les enchaîne. À tout moment, vous savez ce qui marche.

1 Espace · 2 Stockage · 3 Source · 4 Dataflow · 5 Copy · 6 Silver · 7 ML · 8 Gold · 9 Pipeline · 10 Temps réel · 11 Modèle sémantique · 12 Contrôle et arrêt

FONDATIONS

1 Espace de travail et dossiers

Un espace dédié, rangé comme une équipe de production.

1. **Espaces de travail → + Nouvel espace de travail**, saisissez le nom, puis **Avancé → Mode de licence : Évaluation** (ou votre capacité Fabric) → **Appliquer**.
2. Dans l'espace : **Nouveau → Dossier**, cinq fois.

CHAMP	VALEUR À SAISIR
Espace de travail	`RailPulse_Fabric`
Dossiers	`01_Stockage · 02_Ingenierie · 03_Science · 04_Temps_Reel · 05_Restitution`

ONELAKE

2 Deux lakehouses et un entrepôt

Ouvrez 01_Stockage avant de créer : les éléments y naissent directement.

1. **+ Nouvel élément → Lakehouse**. Si la case **Schémas de lakehouse** apparaît, **décochez-la**.
2. Recommencez pour le second lakehouse, puis **+ Nouvel élément → Entrepôt**.

CHAMP	VALEUR À SAISIR
Lakehouse 1 · lakehouse 2	`LH_RailPulse_Bronze · LH_RailPulse_Silver`
Entrepôt	`WH_RailPulse_Gold`

POURQUOI C'EST FAIT AINSI

Bronze garde la donnée telle qu'elle arrive, Silver la certifie en Delta, Gold la sert en T-SQL. Trois responsabilités, trois éléments : un incident sur une couche ne contamine pas les autres. Sans schéma de lakehouse, le code adresse chaque table par `lakehouse.table`, sans ambiguïté.

DATA ENGINEERING · NOTEBOOK

3 NB_01 : le système source simulé

Dossier 02_Ingenierie. Douze mois de données réalistes, défauts compris.

1. Importez `NB_01_Source_Simulator.ipynb`.
2. Panneau Explorateur : **Ajouter des éléments de données → Lakehouse existant → LH_RailPulse_Bronze**. Il devient le lakehouse **par défaut** (épinglé) : c'est là que les fichiers sont déposés.
3. **Exécuter tout**.

Le réseau
12 gares géolocalisées, 12 lignes avec distance, durée, fréquence et prix au kilomètre, 40 rames.

L'exploitation
Un train par ligne : retards selon l'heure de pointe, la météo et l'âge de la rame, fêtes et été.

Les ventes
Réservations par train, classe et canal (App, Web, Gare, Partenaire), délai d'achat.

Les pièges
400 doublons, 40 retards impossibles, 25 gares inconnues : Silver devra les intercepter.

✓ Point de contrôle · après NB_01

- Sortie : `Circulations déposées 98 950 (dont 400 doublons et 65 lignes défectueuses)`.
- **Files/landing** : 4 dossiers (reference, trips avec 12 CSV, bookings avec 12 Parquet, plan).
- **Tables** : bronze_stations (12) et bronze_weather (4 548).

DATA FACTORY · DATAFLOW GEN2

4 DF_RailPulse_Reference : le référentiel sans code

Dossier 02_Ingenierie. Les gares et la météo passent en Silver.

1. **+ Nouvel élément → Dataflow Gen2**, nom, **Créer**. **Obtenir des données** → catalogue OneLake → **LH_RailPulse_Bronze** → cochez **bronze_stations** et **bronze_weather** → **Créer**.
2. Renommez chaque requête (clic droit → Renommer), puis **Ajouter une colonne → Colonne personnalisée** avec la formule ci-dessous.
3. Pour chaque requête : **Destination des données → Lakehouse → LH_RailPulse_Silver → Nouvelle table**, même nom que la requête, méthode **Remplacer**, **Enregistrer les paramètres**. Puis **Enregistrer et exécuter**.

CHAMP	VALEUR À SAISIR
Nom du Dataflow	`DF_RailPulse_Reference`
bronze_stations renommée en · colonne (Texte)	`silver_stations · country_name`
bronze_weather renommée en · colonne (Texte)	`silver_weather · weather_label`

FORMULE COUNTRY_NAME

```
if [country_code] = "FR" then "France" else if [country_code]
= "LU" then "Luxembourg"
else if [country_code] = "BE" then "Belgique" else if
[country_code] = "NL" then "Pays-Bas"
else if [country_code] = "DE" then "Allemagne" else if
[country_code] = "CH" then "Suisse" else "Italie"
```

FORMULE WEATHER_LABEL

```
if [severity] = 0 then "Calme" else if [severity] = 1 then
"Perturbé"
else if [severity] = 2 then "Dégradé" else "Sévère"
```

✓ **Point de contrôle · après le Dataflow**

- LH_RailPulse_Silver : **silver_stations** (12 lignes, country_name) et **silver_weather** (4 548 lignes, weather_label).

DATA FACTORY · PIPELINE

5 PL_RailPulse_Master, première version

Dossier 02_Ingenierie. Les fondations du pipeline, en deux activités.

1. **+ Nouvel élément → Pipeline**. **Activités → Notebook** : nom, puis onglet Paramètres → NB_01_Source_Simulator.
2. **Activités → Copier des données → Ajouter au canevas**. **Source** : LH_RailPulse_Bronze, racine **Files**, **Chemin de fichier générique**, format **DelimitedText**, **Première ligne comme en-tête**. **Destination** : LH_RailPulse_Bronze, racine **Tables**, nouvelle table, action **Remplacer**.
3. Reliez la coche verte du notebook au Copy. **Enregistrer**, **Exécuter**.

CHAMP	VALEUR À SAISIR
Pipeline	`PL_RailPulse_Master`
Notebook · Copy	`NB_Source_Simulator · CP_Trips_To_Bronze`
Source : dossier · fichier génériques	`landing/trips · *.csv`
Destination : table	`bronze_trips`

✓ **Point de contrôle · après l'exécution**

- Deux activités vertes. **bronze_trips** contient **98 950** lignes : douze fichiers en une copie.
- Le chemin générique avalera aussi les fichiers de demain, sans modification. Le Copy ne transforme rien : Bronze reste rejouable.

DATA ENGINEERING · SPARK

6 NB_02 : Bronze vers Silver, la donnée certifiée

Dossier 02_Ingenierie. Qualité, quarantaine, MERGE Delta, V-Order.

1. Importez `NB_02_Bronze_To_Silver.ipynb`, ajoutez **LH_RailPulse_Bronze** et **LH_RailPulse_Silver** (chaque table est nommée avec son lakehouse : l'ordre d'ajout ne compte pas). **Exécuter tout**.

NB_02 · LES CINQ GESTES CLÉS (EXTRAIT)

```
checked = typed.withColumn("dq_reason",
    F.when(~F.col("origin").isin(codes) | ~F.col("destination").isin(codes), "GARE_INCONNUE")
     .when(F.col("arrival_delay_min") < -30, "RETARD_IMPOSSIBLE"))
quarantine.write.mode("overwrite").saveAsTable(f"{SILVER}.silver_quarantine")      # rien n'est jeté en silence
(DeltaTable.forName(spark, target).alias("t")                                      # rejouable sans doublon
    .merge(trips.alias("s"), "t.trip_id = s.trip_id")
    .whenMatchedUpdateAll().whenNotMatchedInsertAll().execute())
silver_bk = bronze_bookings.join(valid_ids, "trip_id", "left_semi")                # aucune réservation orpheline
spark.sql(f"OPTIMIZE {SILVER}.silver_trips VORDER")                                # lecture Direct Lake rapide
```

✓ **Point de contrôle · après NB_02**

- `Bronze : 98 950 | après dédoublonnage : 98 550 | quarantaine : 65 | Silver : 98 485`
- `Réservations Bronze : 781 102 | Silver : 780 586 | orphelines écartées : 516`
- Relancez la cellule 1 : Silver reste à **98 485**. Le MERGE est idempotent.

DATA SCIENCE · MLFLOW

7 NB_03 : prévoir les retards des 14 prochains jours

Dossier 03_Science. Entraîner, tracer, versionner, scorer.

1. Importez `NB_03_ML_Delay_Forecast.ipynb`, ajoutez **LH_RailPulse_Silver**, **Exécuter tout**.
2. L'expérience et le modèle apparaissent dans l'espace : rangez-les dans 03_Science.

NB_03 · L'ESSENTIEL (EXTRAIT)

```
mlflow.set_experiment("EXP_RailPulse_Delay")
model = HistGradientBoostingRegressor(loss="absolute_error", max_iter=300).fit(train[FEATURES], train.label)
mlflow.sklearn.log_model(model, "model", signature=signature, registered_model_name="ML_RailPulse_Delay")
scorer = MLFlowTransformer(inputCols=FEATURES, outputCol="predicted_delay_min",
                           modelName="ML_RailPulse_Delay", modelVersion=version)   # PREDICT natif de Fabric
forecast = scorer.transform(plan)            # risque ÉLEVÉ dès 6 min, MOYEN dès 3 min
```

✓ **Point de contrôle · après NB_03**

- `Entraînement : 81 545 trains | test (août-septembre 2026) : 16 366 trains`
- MAE proche de **2,6 min** contre **3,4** sans modèle : environ **22 %** d'erreur en moins.
- **EXP_RailPulse_Delay**, **ML_RailPulse_Delay** (version 1) et **ml_delay_forecast** (3 780 trains) existent.

POURQUOI C'EST FAIT AINSI

Le découpage est temporel, jamais aléatoire : on apprend sur le passé et on teste sur les deux derniers mois, comme en production. La comparaison à une référence naïve prouve la valeur du modèle au lieu de l'affirmer, et chaque exécution crée une version tracée.

DATA WAREHOUSE · T-SQL

8 WH_RailPulse_Gold : le schéma en étoile

Six requêtes, une par une, dans l'ordre. Script complet : WH_RailPulse_Gold.sql.

1. Ouvrez WH_RailPulse_Gold. Explorateur : **+ Entrepôts → LH_RailPulse_Silver → Confirmer**.
2. **Nouvelle requête SQL** pour chaque script ci-dessous, **Exécuter**. Les vues viennent après le premier chargement : une vue ne peut pas viser une table qui n'existe pas encore.

REQUÊTE 1 · TABLES DE CONTRÔLE ET TABLE D'ANCRAGE DES MESURES (UNE SEULE FOIS)

```
-- RAILPULSE | WH_RailPulse_Gold | Script 1 : tables de contrôle
CREATE TABLE dbo.cfg_emission_factor (
    mode_code     varchar(10)  NOT NULL,
    mode_label    varchar(40)  NOT NULL,
    g_co2_per_pkm decimal(9,2) NOT NULL
);
INSERT INTO dbo.cfg_emission_factor VALUES
    ('TRAIN', 'Train à grande vitesse', 6.00),
    ('AVION', 'Avion court-courrier', 180.00);

CREATE TABLE dbo.etl_run_log (
    run_id        varchar(64)  NOT NULL,
    step_name     varchar(60)  NOT NULL,
    rows_loaded   bigint       NULL,
    logged_at     datetime2(6) NOT NULL
);

-- Table d'ancrage des mesures DAX : une ligne, une colonne masquée dans le modèle sémantique
CREATE TABLE dbo.mesures (valeur int NOT NULL);
INSERT INTO dbo.mesures VALUES (1);
```

REQUÊTE 2 · PROCÉDURE DBO.USP_LOAD_GOLD (COLLEZ LE SCRIPT 2 DU FICHIER)

```
CREATE OR ALTER PROCEDURE dbo.usp_Load_Gold @run_id
varchar(64)
AS BEGIN
  -- facteurs d'émission lus dans cfg_emission_factor
  -- 4 dimensions : dim_date (457 jours, libellé « oct. 25 »),
  --   dim_station, dim_route (24 relations), dim_trainset
  -- 3 faits : fact_trip, fact_booking, fact_delay_forecast
  DROP TABLE IF EXISTS dbo.fact_trip;
  CREATE TABLE dbo.fact_trip AS
  SELECT t.trip_id, ...
  FROM LH_RailPulse_Silver.dbo.silver_trips t   -- lecture
sans copie
  LEFT JOIN (...) p ON p.trip_id = t.trip_id;
  -- ... puis 3 lignes dans dbo.etl_run_log
END;
```

REQUÊTE 3 · PREMIER CHARGEMENT

```
EXEC dbo.usp_Load_Gold @run_id = 'manuel-01';
```

REQUÊTE 4 · VUE DES INDICATEURS PHARES

```
-- RAILPULSE | Script 3 : vues de lecture
CREATE OR ALTER VIEW dbo.vw_headline AS
SELECT COUNT_BIG(*)
AS trains,
       SUM(passengers)
AS passagers,
       SUM(revenue_eur)
AS chiffre_affaires_eur,
       CAST(100.0 * SUM(CAST(is_on_time AS int)) /
NULLIF(SUM(CASE WHEN is_cancelled = 0 THEN 1 ELSE 0 END), 0)
AS decimal(5,2)) AS ponctualite_pct,
       CAST(SUM(co2_avoided_kg) / 1000 AS decimal(18,0))
AS co2_evite_tonnes
FROM dbo.fact_trip;
```

REQUÊTE 6 · CONTRÔLE (LA REQUÊTE 5, VW_LINE_SCORECARD, EST DANS LE FICHIER)

```
SELECT * FROM dbo.vw_headline;
SELECT * FROM dbo.etl_run_log;
```

✓ Point de contrôle · après le chargement

- vw_headline : **98 485** trains, **26 273 478** voyageurs, **1 049 780 452,66** €, **75,56** % à l'heure, **1 164 305** t de CO_2 évitées.
- etl_run_log : 3 lignes pour `manuel-01`. Tables Gold : 4 dimensions, 3 faits, et `mesures` (1 ligne).

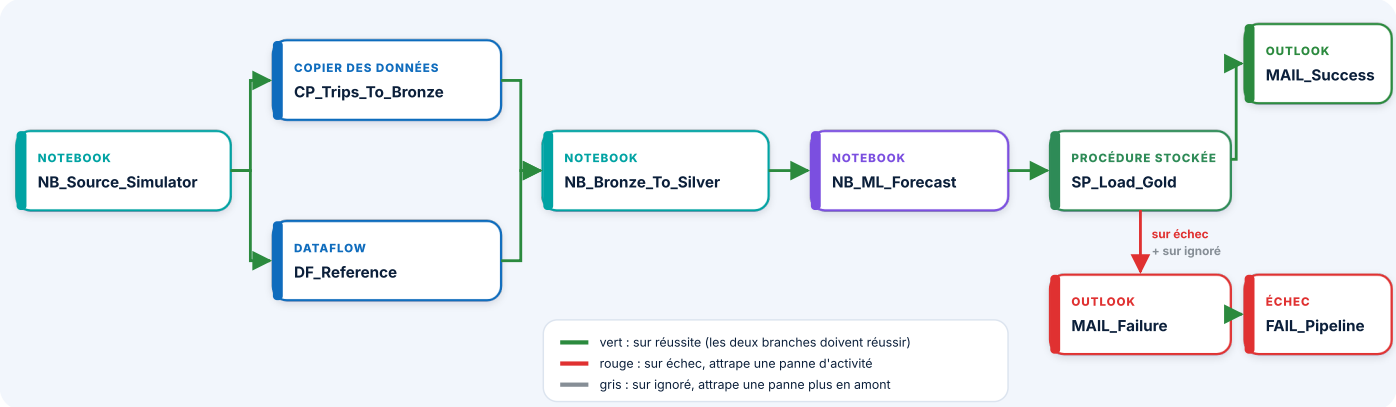
POURQUOI C'EST FAIT AINSI

Les noms en trois parties `LH_RailPulse_Silver.dbo.table` lisent le lakehouse sans le copier. Les facteurs d'émission vivent dans une table de configuration : changez une hypothèse, relancez, tout le CO_2 suit. Le libellé de mois `month_label` et la table `mesures` préparent dès maintenant un modèle sémantique propre.

DATA FACTORY · ORCHESTRATION

9 PL_RailPulse_Master, version complète

Sept activités de plus, une seule exécution de bout en bout.



1. **Dataflow** DF_Reference depuis la coche verte de NB_Source_Simulator, en parallèle du Copy.
2. **Notebook** NB_Bronze_To_Silver, relié aux coches vertes du Copy **et** du Dataflow. Puis NB_ML_Forecast.
3. **Procédure stockée** SP_Load_Gold : entrepôt WH_RailPulse_Gold, procédure, **Importer** les paramètres, valeur en contenu dynamique.
4. **Office 365 Outlook** MAIL_Success sur la coche verte ; MAIL_Failure sur les flèches **rouge (échec)** et **grise (ignoré)** de SP_Load_Gold ; **Échec** FAIL_Pipeline après MAIL_Failure. **Valider**, **Enregistrer**, **Exécuter**.

Avant d'exécuter · Arrêtez toutes les sessions Spark interactives (bouton **Arrêter la session** dans NB_01, NB_02, NB_03) : une capacité d'essai limite les sessions simultanées et une session ouverte bloque les notebooks du pipeline (erreur 430). Sur chaque activité Notebook, onglet Général : **Nouvelle tentative 2**, **intervalle 120 secondes**.

CHAMP	VALEUR À SAISIR
Notebooks : nom · élément	`NB_Bronze_To_Silver · NB_02 ; NB_ML_Forecast · NB_03`
Procédure : nom · paramètre (String)	`SP_Load_Gold · dbo.usp_Load_Gold · run_id`
Valeur de run_id (dynamique)	`@pipeline().RunId`
Objet succès · objet échec	`RailPulse : chargement réussi · RailPulse : échec du chargement`
Corps succès (dynamique)	`@concat('Chargement RailPulse terminé. Exécution ', pipeline().RunId, ' le ', utcNow())`
Corps échec (dynamique)	`@concat('Le pipeline RailPulse a échoué. Exécution ', pipeline().RunId)`
Échec : message · code	`@concat('RailPulse en échec, exécution ', pipeline().RunId) · RP-500`

✓ Point de contrôle · après l'exécution complète

- Sept activités vertes ; MAIL_Failure et FAIL_Pipeline ignorées, c'est normal. Le courriel de succès arrive.
- etl_run_log a 3 lignes de plus avec le RunId. vw_headline n'a pas bougé : toute la chaîne est idempotente.

POURQUOI C'EST FAIT AINSI

Plusieurs conditions venant de la même activité se combinent en OU : « échec » plus « ignoré » sur la dernière activité attrape une panne survenue n'importe où en amont, avec une seule activité de secours. Le Fail final garde le statut rouge dans le hub de surveillance.

REAL-TIME INTELLIGENCE

10 Le flux des trains en direct

Dossier 04_Temps_Reel. L'ordre compte : les événements doivent couler avant la destination.

1. **+ Nouvel élément → Eventhouse** (sa base KQL du même nom est créée avec lui).
2. **+ Nouvel élément → Eventstream → Utiliser un point de terminaison personnalisé**, nom de la source, **Ajouter**, **Publier**.
3. Cliquez la source → **Clés** → protocole **Event Hub** → copiez la **chaîne de connexion de la clé primaire**.
4. Importez `NB_04_Live_Simulator.ipynb`, collez la chaîne entre les guillemets, **Exécuter tout** : 30 minutes de trains en circulation.
5. Eventstream : **Modifier → Ajouter une destination → Eventhouse**, mode **Traitement des événements avant ingestion**, base EH_RailPulse, **Créer une table**, format **JSON**, **Enregistrer**, **Publier**.
6. Base EH_RailPulse → **Interroger avec du code** : exécutez les trois contrôles du fichier KQL, un bloc à la fois.

CHAMP	VALEUR À SAISIR
Eventhouse · Eventstream	`EH_RailPulse` · `ES_RailPulse_Live`
Source · destination	`src_trains` · `dst_eventhouse`
Notebook · table KQL	`NB_04_Live_Simulator` · `live_telemetry`

✓ Point de contrôle · flux en marche

- `live_telemetry | count` augmente à chaque exécution.
- Quelques minutes après le départ, les trains de la ligne L02 passent en INCIDENT.

CONTRÔLE 2 · LE RÉSEAU EN DIRECT

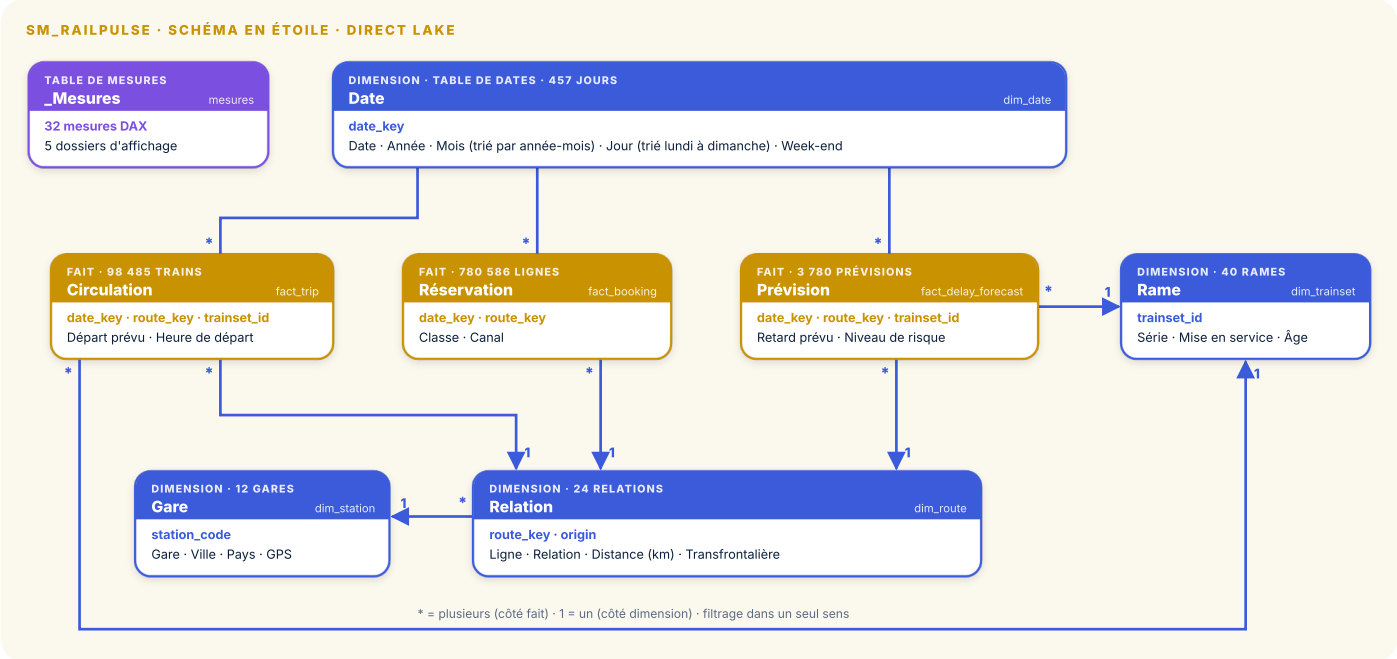
```
live_telemetry
| extend ts = todatetime(event_time)
| where ts > ago(10m)
| summarize trains = dcount(train_id), retard_moyen =
round(avg(delay_min), 1) by line_code
| order by retard_moyen desc
```

Bonus OneLake · Base EH_RailPulse → **Disponibilité OneLake → Activer**, puis dans LH_RailPulse_Silver : **Nouveau raccourci → Microsoft OneLake → EH_RailPulse → live_telemetry**. Le temps réel devient lisible en Spark et en T-SQL, sans copie.

POWER BI · DIRECT LAKE

11 SM_RailPulse : le modèle sémantique

Dossier 05_Restitution. Le point d'arrivée de la plateforme : un modèle prêt pour tout rapport.



A. Créer le modèle et ses relations

1. Ouvrez WH_RailPulse_Gold → onglet **Création de rapports** → **Nouveau modèle sémantique**. Nommez-le, cochez les **huit tables** ci-dessous (aucune vue), **Confirmer**. Le modèle s'ouvre en modification ; sinon, ouvrez-le puis **Ouvrir le modèle de données**.
2. Renommez les tables (double-clic sur le nom, ou volet **Propriétés → Nom**).
3. Relations : faites glisser la colonne du fait sur la colonne de la dimension, vérifiez **Plusieurs à un (*:1)** et **filtrage unique**, **Enregistrer**. Neuf relations, listées ci-dessous.
4. Table Date : clic droit → **Marquer comme table de dates** → colonne **date**.

TABLE GOLD COCHÉE	NOM DANS LE MODÈLE
`dim_date`	**Date**
`dim_route`	**Relation**
`dim_station`	**Gare**
`dim_trainset`	**Rame**
`fact_trip`	**Circulation**
`fact_booking`	**Réservation**
`fact_delay_forecast`	**Prévision**
`mesures`	**_Mesures**

DU FAIT (PLUSIEURS)	VERS LA DIMENSION (UN)
`Circulation[date_key]`	`Date[date_key]`
`Circulation[route_key]`	`Relation[route_key]`
`Circulation[trainset_id]`	`Rame[trainset_id]`
`Réservation[date_key]`	`Date[date_key]`
`Réservation[route_key]`	`Relation[route_key]`
`Prévision[date_key]`	`Date[date_key]`
`Prévision[route_key]`	`Relation[route_key]`
`Prévision[trainset_id]`	`Rame[trainset_id]`
`Relation[origin]`	`Gare[station_code]`

B. Des colonnes lisibles par un humain

Renommez, masquez (sélection multiple avec Ctrl, puis **Propriétés → Masqué**), et triez (**Propriétés → Trier par colonne**). Faites-le **avant** l'étape C : les mesures utilisent les nouveaux noms.

TABLE	COLONNES À RENOMMER (ANCIEN NOM → NOUVEAU NOM)	À MASQUER · TRI · CATÉGORIE
Date	date → `Date` · year → `Année` · month_label → `Mois` · weekday_name → `Jour` · is_weekend → `Week-end`	masquer date_key, month_num, month_name, year_month, weekday_num · Mois trié par `year_month` · Jour trié par `weekday_num`
Relation	line_code → `Ligne` · route_name → `Relation` · distance_km → `Distance (km)` · is_cross_border → `Transfrontalière`	masquer route_key, origin, destination
Gare	station_code → `Code gare` · station_name → `Gare` · city → `Ville` · country_name → `Pays` · latitude → `Latitude` · longitude → `Longitude`	masquer country_code, is_hub · catégorie de données Latitude et Longitude
Rame	trainset_id → `Rame` · series → `Série` · commissioning_year → `Mise en service` · trainset_age → `Âge (ans)`	aucune
Circulation	scheduled_departure → `Départ prévu` · dep_hour → `Heure de départ`	masquer toutes les autres colonnes
Réservation	travel_class → `Classe` · channel → `Canal`	masquer toutes les autres colonnes
Prévision	scheduled_departure → `Départ prévu` · weather_severity → `Sévérité météo` · load_factor → `Remplissage prévu` · predicted_delay_min → `Retard prévu (min)` · risk_level → `Niveau de risque` · model_version → `Version du modèle`	masquer trip_id, date_key, route_key, trainset_id
_Mesures	aucune	masquer valeur : la table devient une table de mesures

C. Les 32 mesures en une seule requête

1. Ouvrez la vue **Requête DAX** du modèle, collez le contenu de `SM_RailPulse_Mesures.dax`, **Exécuter** : la grille affiche les cinq chiffres de contrôle.
2. Cliquez **Mettre à jour le modèle avec les modifications** : les 32 mesures rejoignent la table _Mesures. Si le bouton est absent, utilisez le lien **Mettre à jour le modèle : ajouter une nouvelle mesure** au-dessus de chaque mesure.

3. Mettez en forme (sélection multiple, **Propriétés → Format**) selon le tableau, et, si vous le souhaitez, rangez-les dans les dossiers d'affichage 01 Volumes, 02 Qualité de service, 03 Finance, 04 Climat, 05 Prévisions.

SM_RAILPULSE_MESURES.DAX · DÉBUT DU SCRIPT (32 MESURES DANS LE FICHIER)

```
    // ---- 01 Volumes ----
    MEASURE '_Mesures'[Trains] = COUNTROWS ( Circulation )
    MEASURE '_Mesures'[Trains assurés] = CALCULATE ( [Trains], Circulation[is_cancelled] = FALSE () )
    MEASURE '_Mesures'[Trains annulés] = CALCULATE ( [Trains], Circulation[is_cancelled] = TRUE () )
    MEASURE '_Mesures'[Voyageurs] = SUM ( Circulation[passengers] )
    MEASURE '_Mesures'[Voyageurs-km] = SUM ( Circulation[passenger_km] )
    MEASURE '_Mesures'[Sièges offerts] = CALCULATE ( SUM ( Circulation[seats] ), Circulation[is_cancelled] = FALSE () )
    MEASURE '_Mesures'[Taux de remplissage %] = DIVIDE ( [Voyageurs], [Sièges offerts] )
    MEASURE '_Mesures'[Voyageurs mois précédent] = CALCULATE ( [Voyageurs], DATEADD ( 'Date'[Date], -1, MONTH ) )
    MEASURE '_Mesures'[Évolution voyageurs %] =
        VAR _prec = [Voyageurs mois précédent]
        RETURN IF ( NOT ISBLANK ( _prec ), DIVIDE ( [Voyageurs] - _prec, _prec ) )
    // ... 02 Qualité de service, 03 Finance, 04 Climat, 05 Prévisions, 99 Mise en forme : voir le fichier
EVALUATE ROW ( "Voyageurs", [Voyageurs], "Chiffre d'affaires (M€)", [Chiffre d'affaires (M€)], ... )
```

FORMAT	MESURES CONCERNÉES
Nombre entier, séparateur de milliers	`Trains, Voyageurs, Sièges, CO2 évité (t)…`
Pourcentage, 1 décimale	`Ponctualité %, Taux de remplissage %, Part…`
Décimal, 1 décimale	`Retard moyen, Écart (pts), CA (M€), Anticipation`
Monétaire €	`Chiffre d'affaires, Recette par voyageur (-km)`

✓ Point de contrôle · modèle prêt

- La requête DAX affiche : **26 273 478** voyageurs · **1 049,78** M€ · ponctualité **0,7556** · **1 164 305** t · **3 780** trains prévus.
- Volet Données : 7 tables métier, la table _Mesures en tête avec son icône de calculatrice, aucune colonne technique visible.

POURQUOI C'EST FAIT AINSI

Direct Lake lit les fichiers Delta de OneLake sans import ni actualisation planifiée : le modèle voit chaque chargement du pipeline. Il pointe les tables et jamais les vues, qui le feraient basculer en DirectQuery. Les noms de tables accentués sont écrits entre apostrophes dans le DAX (`'Réservation'`), et la mesure d'objectif ne renvoie rien sur une période sans train, pour ne jamais créer de ligne vide dans un visuel.

CONTRÔLE FINAL ET SOBRIÉTÉ

12 Vérifier, dépanner, puis tout arrêter

Une plateforme bien conçue ne consomme rien quand personne ne la regarde.

Contrôle final

1. L'espace contient 15 éléments (la base KQL compte) rangés dans leurs dossiers.
2. **Vue de lignage** : la chaîne source → Bronze → Silver → Gold → SM_RailPulse apparaît de bout en bout.
3. **Hub de surveillance** : PL_RailPulse_Master et ses activités sont vertes.
4. Tous les chiffres de la page « Les règles du jeu » sont retrouvés.

Arrêter la consommation

1. Simulateur : il s'arrête seul après 30 minutes ; puis **Arrêter la session** de chaque notebook.
2. ES_RailPulse_Live : désactivez la source **src_trains**.
3. Pipeline planifié : **Planifier → Désactivé**.
4. Capacité payante : mettez-la en pause dans le portail Azure.

Si ça bloque

SYMPTÔME	CAUSE	CORRECTION
NB_01 : « Lakehouse par défaut = ... »	Un autre lakehouse est épinglé par défaut.	Épinglez LH_RailPulse_Bronze comme lakehouse par défaut, relancez.
Table introuvable, tables rangées sous dbo	Lakehouse créé avec **Schémas de lakehouse** coché.	Recréez le lakehouse sans cette option.
Notebook du pipeline en erreur 430	Trop de sessions Spark ouvertes sur la capacité d'essai.	Arrêtez les sessions interactives ; gardez 2 nouvelles tentatives à 120 s.
Vue SQL refusée : objet introuvable	La vue a été créée avant le premier chargement.	Exécutez la requête 3 (EXEC), puis les vues.
Destination Eventhouse sans colonnes	Aucun événement ne coulait au moment de l'ajout.	Lancez NB_04, attendez une minute, puis ajoutez la destination.
Requête DAX : « Invalid token » sur un accent	Nom de table accentué non protégé.	Écrivez `'Réservation'` et `'Prévision'` entre apostrophes.
Mois triés d'avril à septembre	Tri alphabétique par défaut.	Mois : **Trier par colonne → year_month** ; Jour : **weekday_num**.
Modèle en DirectQuery, plus lent	Une vue SQL a été ajoutée au modèle.	Retirez la vue : le modèle ne garde que des tables.

L'inventaire final de l'espace RailPulse_Fabric

DOSSIER	ÉLÉMENTS	RÔLE
01_Stockage	`LH_RailPulse_Bronze · LH_RailPulse_Silver · WH_RailPulse_Gold`	les trois couches du médaillon
02_Ingenierie	`NB_01 · NB_02 · DF_RailPulse_Reference · PL_RailPulse_Master`	produire, certifier, orchestrer
03_Science	`NB_03 · EXP_RailPulse_Delay · ML_RailPulse_Delay`	prévoir les retards
04_Temps_Reel	`NB_04 · ES_RailPulse_Live · EH_RailPulse (+ base KQL)`	suivre le réseau à la seconde
05_Restitution	`SM_RailPulse`	servir les mesures à tous les rapports

LA SUITE

SM_RailPulse est la fondation de tout ce qui se regarde : rapports Power BI, tableaux de bord en temps réel, applications. Chacun réutilise ces 32 mesures, si bien que tout le monde compte de la même façon : c'est cela, une plateforme de données.